

Práctica de Afianzamiento: Gestor de Catálogo Multi-Formato

Consolidación de Flujos de Texto (CSV/NIO.2), Persistencia Binaria (RAF/Serialización) y Ecosistema XML (DOM/SAX/StAX/JAXB)

MÓDULO PROFESIONAL	CICLO Y CURSO	MODALIDAD Y DURACIÓN	CARÁCTER DE LA TAREA
Acceso a Datos (0486)	DAM 2º Curso · 2026-2027	Presencial / Distancia · 180 min (3 sesiones)	Formativa · Afianzamiento previo a examen

ALINEACIÓN CURRICULAR OFICIAL (REAL DECRETO 455/2010 · CURRÍCULO ASTURIAS)

- RA1: Manejo de Ficheros
- CE1.a (Clases de E/S y Flujos)
- CE1.b (Ficheros de Texto)
- CE1.c (Ficheros Binarios y RAF)
- CE1.d (Serialización de Objetos)
- CE1.e (Procesamiento XML: DOM, SAX, JAXB)

Esta práctica de afianzamiento integra todos los saberes básicos de la Unidad de Trabajo 1. El estudiante desarrolla una aplicación CLI robusta que ingesta datos en texto plano delimitado, optimiza accesos directos en almacenamiento binario y transforma los catálogos a formatos estructurados XML, aplicando un tratamiento exhaustivo de excepciones y control de recursos.

1. Contexto Técnico del Reto

La distribuidora tecnológica **ElectroTech S.L.** se encuentra en pleno proceso de modernización de su plataforma logística. En la actualidad, coexisten tres terminales desconectados que generan y consumen información en formatos incompatibles:

- Terminales Punto de Venta (TPV):** Exportan diariamente ficheros de texto delimitados por comas (`CSV`) con codificación conflictiva (Windows-1252 / ISO-8859-1) que contienen líneas incompletas o corruptas por fallos de conectividad.
- Servidores de Almacén:** Demandan ficheros binarios compactos de acceso aleatorio (`.dat`) para consultar o modificar el stock físico en milisegundos mediante `RandomAccessFile` , sin necesidad de reescribir todo el fichero.
- Pasarelas B2B y Auditoría:** Exigen volcados estructurados en formato `XML` para sincronizar el catálogo con clientes externos y copias de seguridad de objetos Java serializados (`.ser`).

Tu cometido como desarrollador/a de la división de persistencia consiste en construir una herramienta de consola (CLI) en **Java 21** estructurada en paquetes limpios que realice la ingesta, limpieza, conversión, consulta directa y exportación entre estos formatos.

2. Requerimientos Funcionales y Técnicos Paso a Paso

Tarea 1: Ingesta, Validación y Limpieza de Fichero de Texto CSV (2.0 puntos)

- Lectura controlada con Charset:** Leer el archivo `data/productos_import.csv` especificando explícitamente `StandardCharsets.ISO_8859_1` mediante `Files.newBufferedReader()` .
- Validación rigurosa de líneas:**

- Estructura esperada por registro: `id,referencia,categoria,precio,stock` .
- Descartar líneas vacías o que comiencen por comentario (`#` o `;`).
- Validar que la línea posee exactamente 5 campos y que `precio` (double) y `stock` (int) son numéricamente válidos.
- Las líneas erróneas o corruptas deben descartarse y registrarse en `data/errores_csv.log` indicando el número de línea, la causa del error y el contenido original.

3. **Normalización a memoria:** Cargar los productos limpios y válidos en una colección `List<Producto>` en memoria.

Tarea 2: Almacenamiento Binario, RAF y Serialización (3.0 puntos)

- Exportación Binaria Compacta (`DataOutputStream`):** Volcar los productos válidos en `data/catalogo.dat` escribiendo de manera tipada: `writeInt(id)` , `writeUTF(referencia)` , `writeUTF(categoria)` , `writeDouble(precio)` y `writeInt(stock)` .
- Gestor de Stock Directo con `RandomAccessFile`:**
 - Definir registros de longitud fija (ej. 4B id + 20B referencia en chars + 20B categoria en chars + 8B precio + 4B stock = **56 bytes fijos**).
 - Implementar `actualizarStockEnDisco(int posicion, int nuevoStock)` utilizando `raf.seek(offset)` para sobrescribir exclusivamente los 4 bytes del campo stock en disco sin reescribir el resto del archivo.
 - Verificar la posición con `raf.getFilePointer()` y asegurar que no sobrepasa `raf.length()` .
- Backup de Objetos con Serialización (`ObjectOutputStream`):**
 - La clase `Producto` debe implementar `Serializable` con `serialVersionUID = 1L` .
 - Marcar el campo `transient double margenBeneficioInterno` para asegurar que el coste sensible de proveedor no se persiste en el disco.
 - Implementar `backupIncremental(File archivo, Producto prod)` utilizando la técnica `MiObjectOutputStream` para evitar corrupción de cabecera (`StreamCorruptedException`) al realizar anexados (append).

Tarea 3: Catálogo XML y Ecosistema de Parsers (3.0 puntos)

- Generación DOM con Transformer (`DOMSource` y `StreamResult`):**
 - Construir en memoria el árbol DOM mediante `DocumentBuilder.newDocument()` con la raíz `<catalogo fecha="AAAA-MM-DD">` y elementos `<producto id="...">` .
 - Volcar a `data/catalogo.xml` utilizando `Transformer` configurado con sangría de 4 espacios (`OutputKeys.INDENT = "yes"` y `indent-amount = 4`).
- Consultas DOM:** Implementar un método que reciba una categoría (ej. "Informatica"), navegue por el árbol normalizado (`doc.getDocumentElement().normalize()`) ignorando nodos `#text` (`Node.ELEMENT_NODE`) y devuelva el stock total acumulado de dicha categoría.
- Alternativa de Alto Rendimiento (SAX o StAX):** Implementar un analizador reactivo con `DefaultHandler` (SAX) o iterativo con `XMLStreamReader` (StAX) que imprima por consola el número total de productos y el producto con precio más elevado sin cargar el árbol en memoria RAM.

Tarea 4: Arquitectura, Clean Code y Manejo de Excepciones (2.0 puntos)

- Estructura de paquetes profesional:**

```
es.cifpaviles.dam.catalogo/
├─ modelo/           (Producto.java, Categoria.java)
├─ servicio/         (FicheroTextoService, FicheroBinarioService, FicheroXmlService)
├─ excepcion/        (CatalogoCorruptedException, ProductoNotFoundException)
├─ util/             (MiObjectOutputStream.java)
└─ app/              (AppConsola.java - Menú interactivo con Scanner)
```

2. **Uso estricto de `try-with-resources`** en todos los flujos de lectura y escritura para garantizar la liberación inmediata de descriptores de archivo.
3. **Captura selectiva y tipada** de excepciones (`FileNotFoundException` , `EOFException` , `InvalidClassException` , `ParserConfigurationException` , `TransformerException`).

3. Entorno de Desarrollo y Regulación de Inteligencia Artificial

Entorno de Desarrollo de Referencia

El entorno de trabajo principal en el aula es **Eclipse IDE 2024 / 2025** configurado con **Java SE 21 LTS**. Se admite el uso de IntelliJ IDEA o VS Code siempre que se respete la jerarquía estándar de paquetes y dependencias Java nativas.

Política de Uso de Inteligencia Artificial (1ª Evaluación)

En las prácticas y retos formativos de la 1ª Evaluación, las herramientas de IA generativa (ChatGPT, Claude, Copilot) **se permiten exclusivamente para la resolución de dudas teóricas o comprensión de mensajes de error**. No se recomienda su uso para generar el código de la práctica, con el fin de afianzar la destreza mental autónoma requerida en los exámenes presenciales de aula. El docente podrá solicitar la **defensa oral individual** de cualquier fragmento de código entregado.

4. Rúbrica Analítica de Evaluación (Total: 10.0 Puntos)

CRITERIO / PONDERACIÓN	EXCELENTE (100%)	NOTABLE (75%)	APROBADO (50%)	INSUFICIENTE (<50%)
1. Ficheros de Texto y NIO.2 [2.0 Puntos] CE1.a, CE1.b	Lectura con <code>Files.newBufferedReader</code> y <code>StandardCharsets</code> impecable; validación exhaustiva de campos y log de descarte en <code>errores_csv.log</code> con formato riguroso.	Lee y normaliza el CSV correctamente. Detecta líneas erróneas pero el formato del log es básico o no indica número de línea.	Lee el archivo en UTF-8 sin controlar codificaciones conflictivas o interrumpe la ejecución ante registros corruptos sin registrar log.	No utiliza buffers, fallos graves en parsing de cadenas (<code>split</code>) o incapacidad de leer el archivo CSV.
2. Persistencia Binaria, RAF y Serialización [3.0 Puntos] CE1.c, CE1.d	Manejo exacto de tipos con <code>DataStreams</code> ; cálculo matemático impecable de offsets con <code>seek()</code> y registros de 56B en RAF; serialización con <code>serialVersionUID</code> , <code>transient</code> y <code>MIObjectOutputStream</code> sin corrupción.	Escritura binaria y serialización correctas; actualización RAF funcional con pequeñas imprecisiones en el cálculo de longitud o ausencia de <code>transient</code> .	Realiza la escritura binaria pero actualiza registros reescribiendo el archivo completo en lugar de usar <code>seek()</code> posicional directo, o falla en append de objetos.	Ficheros binarios corruptos por orden incorrecto de lectura de tipos primitivos, o no implementa <code>Serializable</code> .
3. Ecosistema XML (DOM, Transformer y SAX/StAX) [3.0 Puntos] CE1.e	Construcción DOM limpia serializada con <code>Transformer</code> sangrado (pretty-print); consultas DOM precisas esquivando nodos <code>#text</code> ; procesador reactivo SAX o Pull StAX funcional con consumo mínimo de RAM.	Genera el XML formateado y ejecuta consultas DOM correctas. El parser SAX/StAX funciona pero acumula datos en memoria o tiene ligeras advertencias.	Genera el XML sin indentación legible y la consulta DOM falla ante saltos de línea al no validar <code>ELEMENT_NODE</code> ; no implementa SAX ni StAX.	Documento XML mal formado, etiquetas no anidadas correctamente o colapso del programa ante parseo de nodos.
4. Arquitectura, Clean Code y Excepciones [2.0 Puntos] CE1.a, Calidad	Estructura modular perfecta en paquetes (modelo, servicio, util, app); uso riguroso de <code>try-with-resources</code> sin fugas de descriptores; menú CLI interactivo claro y captura diferenciada de excepciones.	Código limpio y empaquetado adecuado; gestión segura de flujos; menú funcional con gestión genérica de algunas excepciones (<code>Exception</code>).	Clases concentradas en un único paquete o método <code>main</code> monolítico; cierra los flujos manualmente sin <code>try-with-resources</code> .	Fugas masivas de recursos (ficheros bloqueados en disco), código no compila o excepciones no controladas que provocan volcados de traza en consola.
PUNTUACIÓN TOTAL MÁXIMA:				10.0 PUNTOS

5. Instrucciones de Entrega y Evaluación

El proyecto deberá entregarse a través de la plataforma del curso (Moodle / Google Classroom) o repositorio en GitHub Classroom antes de la fecha límite fijada por el docente:

- **Formato:** Archivo comprimido `Apellido1_Apellido2_Nombre_UT1_Catálogo.zip` o enlace al repositorio Git limpio (sin carpetas `bin/`, `target/` ni `.settings/`).
- **Carpeta `data/`:** Debe incluirse con los ficheros de muestra generados durante las pruebas (`productos_import.csv`, `errores_csv.log`, `catalogo.dat`, `backup_catalogo.ser` y `catalogo.xml`).
- **Memoria Técnica Breve:** Documento Markdown (`README.md`) que describa la arquitectura del proyecto, la fórmula de cálculo del tamaño de registro en `RandomAccessFile` y capturas de consola demostrando la ejecución del menú.